

```
# perl -ne '$s1="a.*"; if($s1/) { print $_ }' file3.txt
aaaa
aabb
aacc
# perl -ne '$s2=".*c$"; if($s2/) { print $_ }' file3.txt
cccc
aacc
bbcc
# perl -ne '$s1="a.*"; $s2=".*c$"; if($s1$s2/) { print $_ }' file3.txt
aacc
```

Figure 5: Regular expressions in Perl variables

the regular expression matches lines of text that end with a c. But what is even more promising is that you can combine these variables to form complex regular expressions. What will be the output of the following command?

```
perl -ne '$s1="a.*"; $s2=".*c$"; if($s1$s2/) { print $_ }' file3.txt
```

Only the string *aacc* is selected because by combining the two variables having regular expressions we will get a match only if a line of text starts with an *a* and ends with a *c*. Figure 5 shows the output of these commands.

It is now time to discuss operator *m//* and *s///* in Perl regular expressions. Now that we have worked with a number of regular expressions, it is very easy to understand the operator *m//*. It is the match operator of Perl and, by default, the regular expression match objects */PATTERN/* and *m/PATTERN/* are the same. Here, *PATTERN* is a regular expression pattern. If you have any doubt, make a similar modification in the script *regex1.pl* where the line of code *if(/UNIX/)* is replaced with *if(m/UNIX/)* as *regex5.pl*, and on testing them, you will see that the results are the same. But if you are new to Perl regular expressions it is better you use the operator *m//* rather than the operator *//* to avoid confusion.

The other two operators of interest are the binding operators *=~* and *!~* which are used to bind a string with the regular expression to be matched. Here again, you don't need to perform an explicit binding if you are processing the current line being read by Perl denoted by *\$_*. So there is no difference between the lines of code *if(\$_ =~ m/UNIX/)* and *if(m/UNIX/)*. You can test the scripts *regex1.pl* and the modified script *regex6.pl* to see that the two give the same output. So it should be clear that the lines of code *if(\$_ =~ m/UNIX/)*, *if(m/UNIX/)*, *if(\$_ =~ /UNIX/)* and *if(/UNIX/)* are all the same.

The real use of binding operators occurs when you want to match a regular expression with a string other than the current line being read by Perl. For example, the command:

```
perl -e '$str="Hello World"; if($str =~ /Hello/) {print "Match found\n"}'
```

... using the binding operator, results in a match and prints the message: 'Match found'. Similarly, the command:

```
perl -e '$str="Hello World"; if($str !~ /abc/) {print "No match found\n"}'
```

...results in no match being found, and the message 'No match found' is printed.

Regular expressions in programming languages often surpass the theoretical limits of actual regular expressions. The operator *s///* used for substituting a matched string with another string is one such example. The general syntax is *s/PATTERN/REPLACEMENT/* where *PATTERN* is a regular expression pattern to be matched and *REPLACEMENT* is the string to be swapped with the matched pattern. For example, consider the Perl one-liner:

```
echo "abc pqr abc" | perl -e '$str = <> ; $str =~ s/abc/xyz/; print "$str"'
```

This will output the message *xyz pqr abc* on the screen. Now let us analyse the command. The output of the *echo* command is read to the Perl script through *STDIN* *<>* and assigned to the variable *\$str*. Then the regular expression substitute object *s/abc/pqr/* substitutes the first occurrence of the word *abc* with *pqr* in the variable *\$str*, and finally the content of the variable *\$str* is printed on the screen. Modifiers like *i* and *m* are used with both the operators *m//* and *s///*, whereas certain other modifiers are applicable only to the operator *s///*. An example of such a modifier is *g*, which results in a global replacement of matched strings, where all the matched strings are replaced with the replacement pattern rather than the first occurrence.

For example:

```
echo "abc pqr abc" | perl -e '$str = <> ; $str =~ s/abc/xyz/g; print "$str"'
```

...will print the message *xyz pqr xyz* on the screen.

Another example is the modifier *r*, which returns the modified string and the original string is left unchanged.

In the example:

```
echo "abc pqr abc" | perl -e '$str = <> ; $str =~ s/abc/xyz/r; print "$str"'
```

...will print *abc pqr abc* because the original string in the variable *\$str* is unchanged, whereas the command:

```
echo "abc pqr abc" | perl -e '$str = <> ; print $str =~ s/abc/xyz/r'
```

...will print the string *xyz pqr abc* on the screen because that is the value returned by the operator *s///*. It is also possible to combine multiple modifiers like *g* and *r*. With that in mind, what will be the output of the following command?

```
echo "abc pqr abc" | perl -e '$str = <> ; print $str =~ s/abc/xyz/gr'
```